4-Bit Counter FSM with Seven-Segment Display

Project Report

Prepared as a project-report template for the 4-Bit-Counter-FSM repository. Confirm the module interfaces, reset behavior, display polarity, and simulation results against the team's actual SystemVerilog implementation before final submission.

1. Abstract

This project implements a 4-bit counter controlled by a finite-state machine (FSM). The counter datapath uses a 2:1 multiplexer to select either the incremented count or the current count, and four flip-flops store the 4-bit value. A hexadecimal-to-seven-segment decoder converts the count into display signals. A top-level module connects the design blocks, while a testbench is used to verify the integrated behavior.

2. Objectives

• Design a 4-bit counter with values represented by count[3:0].

• Use RUN and HOLD control states to enable or pause counting.

• Use a 2:1 MUX to select next-count data.

• Decode hexadecimal values 0–F for a seven-segment display.

• Integrate the modules and verify behavior using a SystemVerilog testbench.

3. System Architecture

Module / artifact	Responsibility
counter_fsm.sv	Counter datapath and RUN/HOLD control logic (confirm exact partition in source).
hex_to_7seg.sv	Maps a 4-bit hexadecimal input to seven-segment outputs.
counter_top.sv	Connects the counter and decoder and exposes top-level I/O.
counter_tb.sv	Applies stimulus and checks the integrated design.

4. FSM Behavior

The intended control model has two states. In RUN, the next-count selection is count + 1. In HOLD, the next-count selection is the current count, so the stored value remains unchanged. With a 4-bit unsigned count, incrementing 1111■ wraps to 0000■. The control convention shown here assumes run = 1 means RUN and run = 0 means HOLD; adapt the diagram if the implemented design uses a different convention.

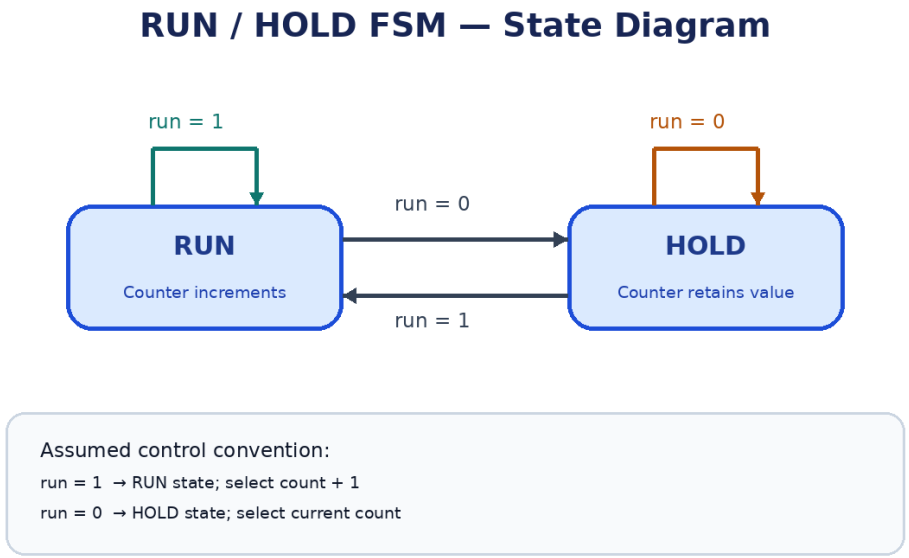


Figure 1. RUN/HOLD state diagram (assumed control convention).

5. Seven-Segment Decoder

The decoder receives a 4-bit value and generates seven output signals for the segments a–g. The truth table should cover all hexadecimal input values from 0 through F. The exact output patterns depend on whether the target display is common-anode or common-cathode and on the segment bit ordering, so those details must match the implementation and the provided truth table.

6. Verification Plan

Test	Expected observation
Reset	Counter enters the implementation's defined initial state.
RUN	Count advances by one on each active clock edge while RUN is selected.
HOLD	Count remains unchanged across clock edges.
Rollover	The 4-bit count wraps from F to 0 if ordinary modulo-16 incrementing is implemented.
Decoder	Seven-segment outputs match the project's truth table for tested values.

7. Tools and Execution

The design is written in SystemVerilog (.sv). One possible open-source simulator is Icarus Verilog with SystemVerilog support. From the repository root, compile the three source modules and the testbench together, then run the generated simulation executable. The final README should document the exact command and the output observed on the team's machine.

8. Conclusion

The project combines sequential counter logic, FSM-based control, a MUX-based next-count path, and hexadecimal display decoding. Successful completion requires the integrated simulation to pass the RUN, HOLD, reset, rollover, and decoder checks. This report is a starter template and should be updated with actual simulation evidence, team member names, course details, and any deviations from the assumed behavior.

Team members: Person 1 — FSM design; Person 2 — counter and MUX design; Person 3 — seven-segment decoder; Person 4 — integration and testbench.